AI-Assisted Literature Review

Using Python + Language Models

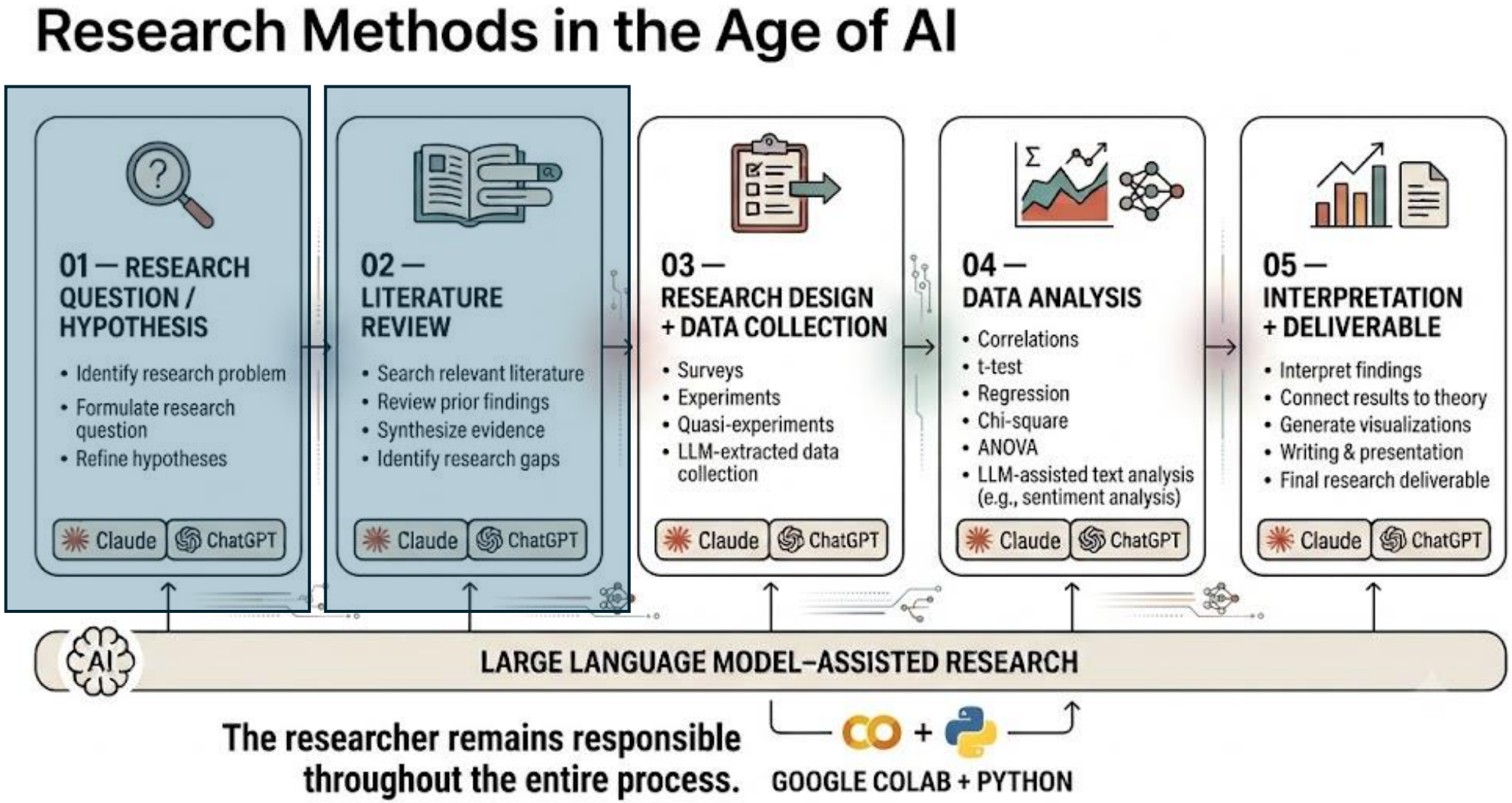
In Previous Weeks, We Learned:

- **Large Language Models (LLMs)** — how they work and how they can help us with research tasks, such as refining research questions
- **Python** — how to write and run basic Python code in Google Colab
- **This Week:** Combine Python + LLMs to help with the next step of research: **Literature Review**

What Is a Literature Review?

- A literature review helps us understand:
  - **What is already known?**
  - **What has been studied?**
  - **What is still missing?**
- Existing research → New research

Where Does Literature Review Fit?

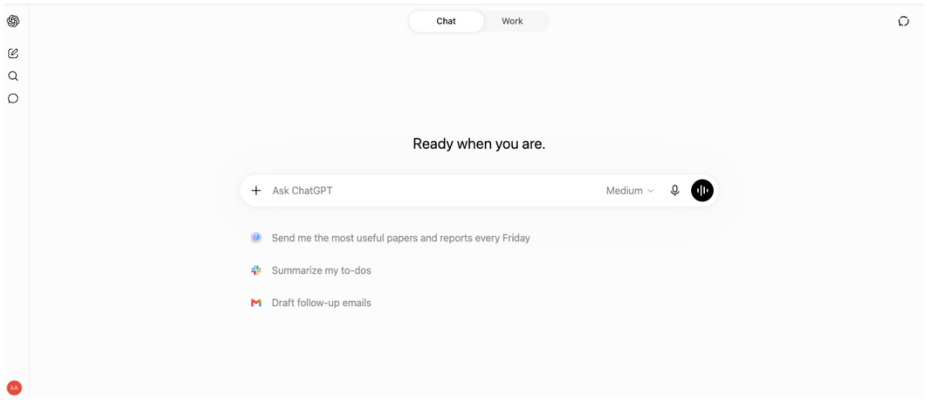


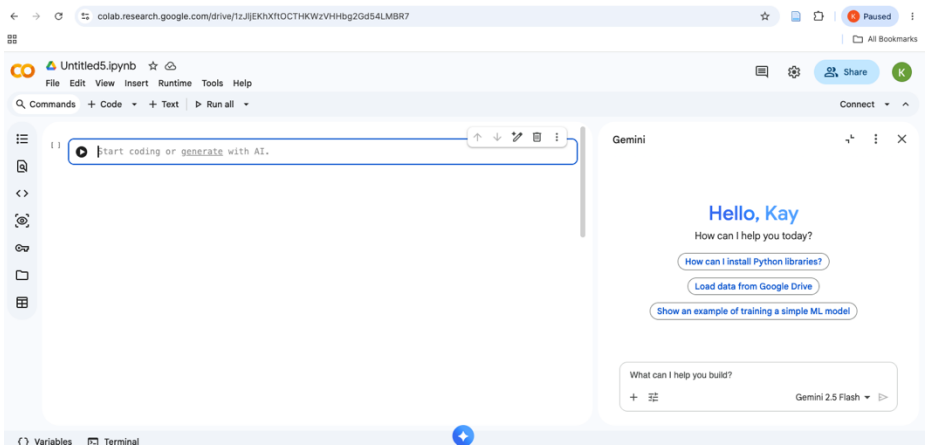
The Challenge: Literature Reviews Can Get Big

- Imagine your Google Scholar search finds **500 articles**
- Which ones are:
  - **Relevant?**
  - **Maybe relevant?**
  - **Not relevant?**

LLMs Can Help Us Review Literature. They can:

- **Screen** abstracts
- **Organize** studies
- **Classify** text
- **Summarize** information
- But: **The researcher makes the final decision.**





Why Python? Why Not Just Use ChatGPT?

- For **3 abstracts, c**hatbot may be easy.
- For **500 abstracts, p**ython can help us:
  - Repeat the **same procedure**
  - Process **many texts**
  - Save the results
  - Repeat the analysis

Remember Last Week?

- Python helped us:

1. STORE

2. FIND

3. REPEAT

4. DECIDE

5. REUSE

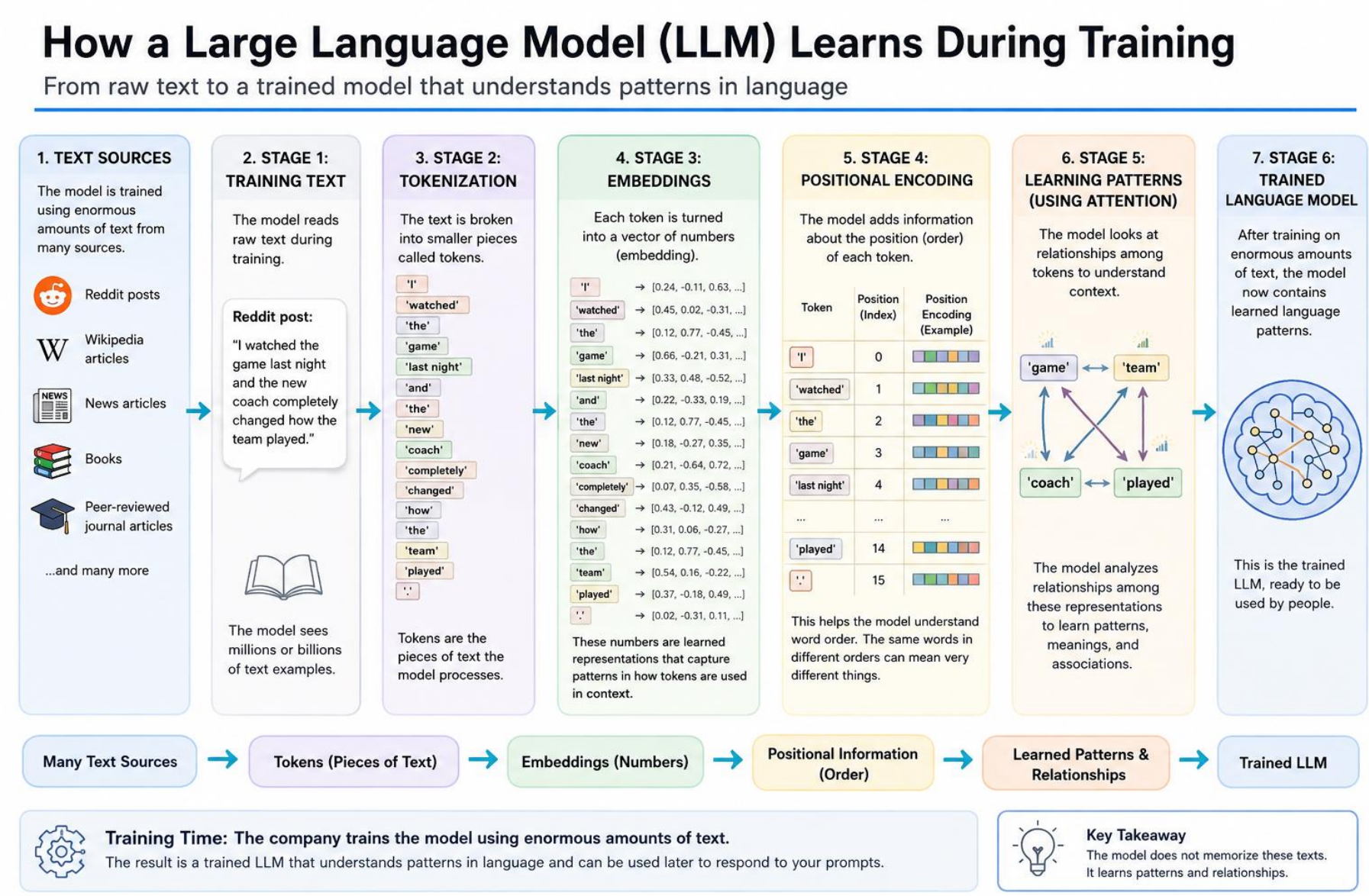
- Today, w**e add an LLM.**

Two Ways Researchers Interact With AI (LLMs)

1. Chat interface: You type into ChatGPT, Claude, or Gemini.
2. API: Your Python code sends text to a model running somewhere else.
3. Pretrained model in Python: Python loads an existing model that we can use directly in our workflow.

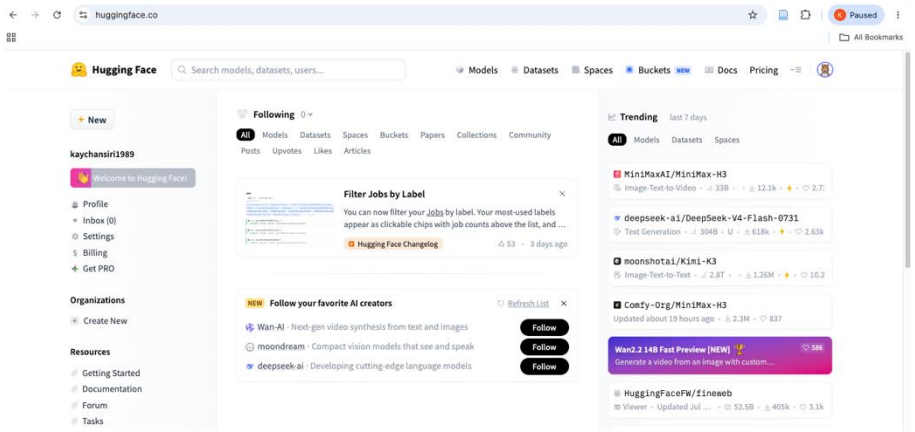
Today, we will practice #3

What is a Pretrained Model?



Where Can We Find Existing Models?

- Hugging Face: Platform hosting models (including transformers), datasets, code
- Model Hub: thousands of pretrained models
- Open-source and proprietary



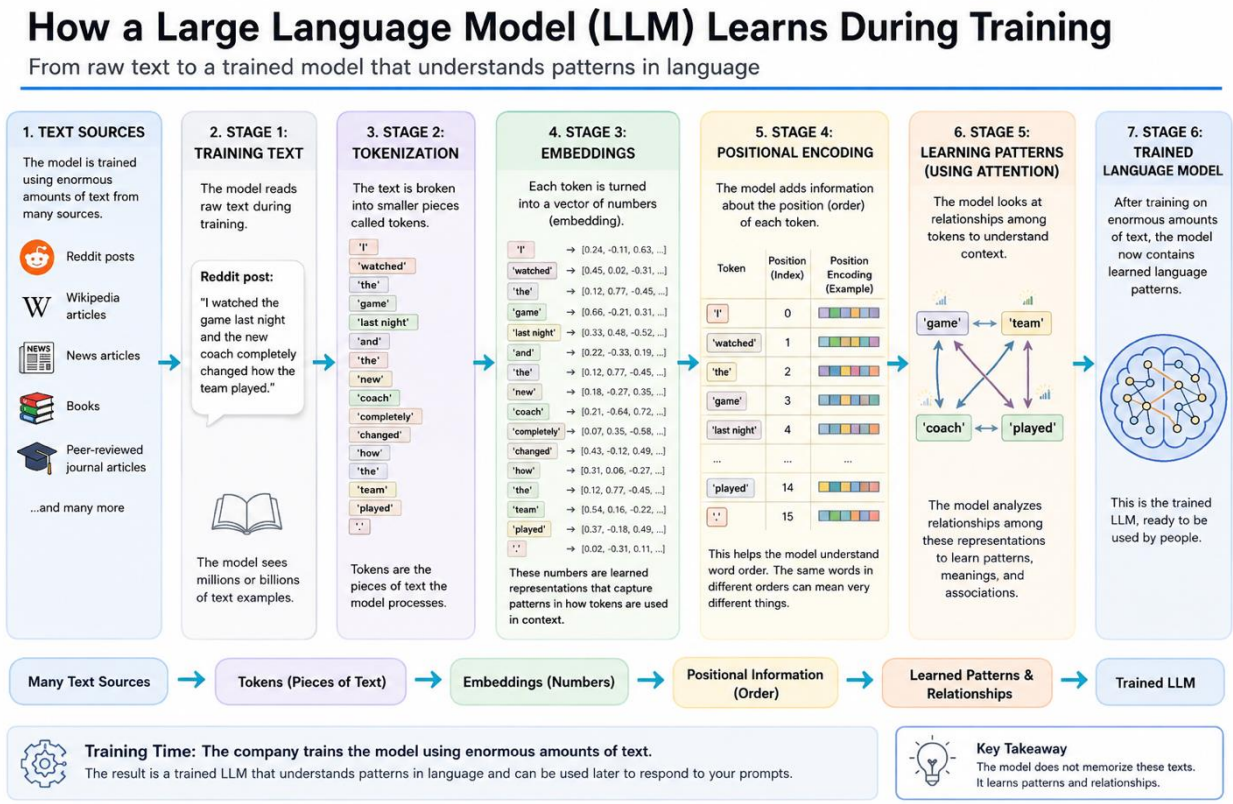
Don’t Just Pick Any Model. Choosing a Model Is a Research Decision.

- ALWAYS read a model card: What does it do?, What data trained it?, What languages?, What limitations?, How well does it perform?
- Do NOT assume every model on Hugging Face is appropriate simply because it can technically process your text.

Before the LLMs-Assisted Literature Review: Familiarize Yourself with Hugging Face's pipeline() Function

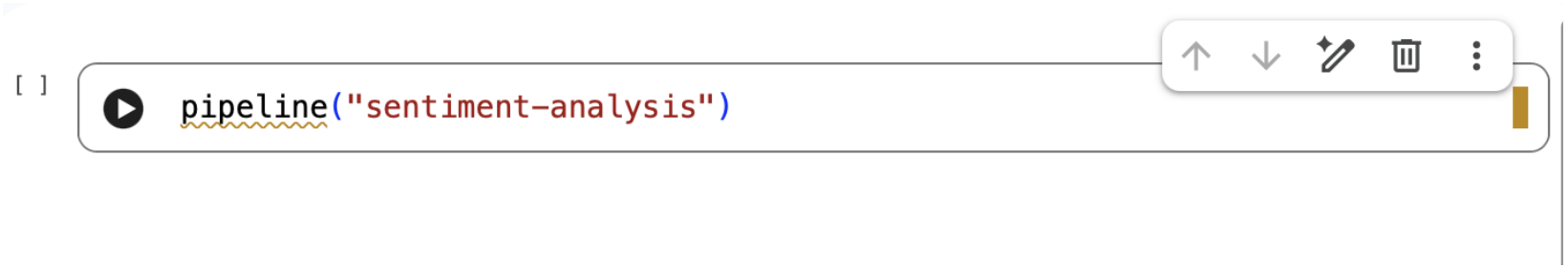
What Is a pipeline()?

- One of the easiest ways to begin using pretrained Transformer models through Python.
- Instead of manually performing every step we learned in Week 2 about the Transformer Architecture, pipeline() connects the pieces required for inference.
- Workflow in pipeline(): Choose a task, load a pretrained model, provide text, and receive readable output



Let’s Start With Sentiment As a Our First Easy Example

- The first argument tells the pipeline **what task we want performed**.



- Different tasks require different model outputs or model heads.

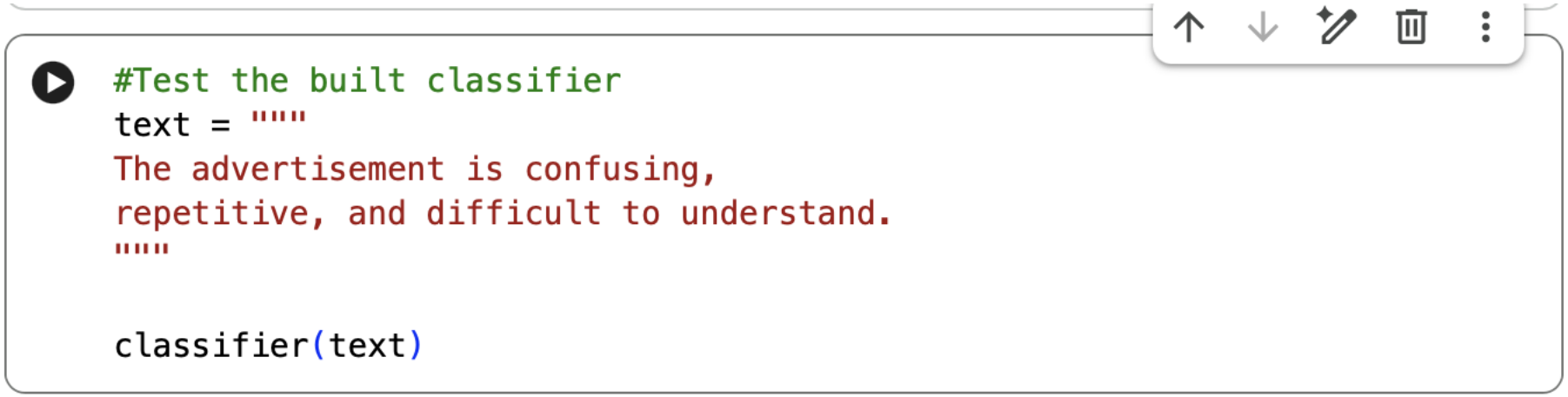
Which Model Does the Pipeline Use?

- Model may be selected automatically
- Model can also be specified
- Model choice matters

Let’s Practice: Choose your model on the Hugging Face Model Hub and create a text

```
2]
from transformers import pipeline

classifier = pipeline(
    "sentiment-analysis",
    model="distilbert-base-uncased-finetuned-sst-2-english"
)

text = "The campaign tells a compelling story and clearly communicates it

result = classifier(text)

print(result)
```

Let’s try another example



```
#Test the built classifier
text = """
The advertisement is confusing,
repetitive, and difficult to understand.
"""

classifier(text)
```

One more example: One Model, Multiple Texts

```
#One model, multiple texts
posts = [
    "The campaign tells a powerful story.",
    "The advertisement is confusing and frustrating.",
    "The video introduces the new product."
]

results = classifier(posts)

print(results)
```

Look at one result. Remember Indexing + Dictionaries?

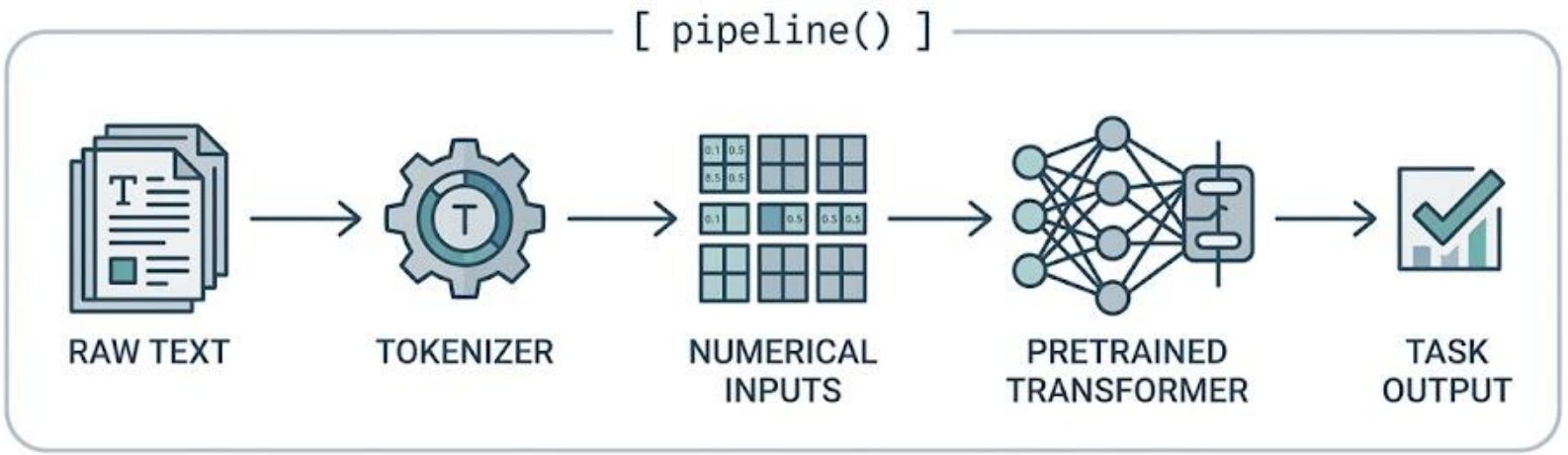
```
#Look at one result
results[0]
```

```
{'label': 'POSITIVE', 'score': 0.9998776912689209}
```

```
#Look at the label of the first results
results[0]["label"]
```

```
'POSITIVE'
```

What Happens Behind pipeline()?



Open the Pipeline: See the Tokenizer

```python
#Behind the pipeline
from transformers import AutoTokenizer

checkpoint = (
    "distilbert-base-uncased-finetuned-sst-2-english"
)

tokenizer = AutoTokenizer.from_pretrained(checkpoint)

text = "News audiences increasingly encounter AI-generated content."

tokens = tokenizer.tokenize(text)

print(tokens)
```

```
['news', 'audiences', 'increasingly', 'encounter', 'ai', '-', 'generated',
```

Remember Tokens?

- **The Model Does Not Read Text Like We Do**
- **Tokens → Numbers**
- Then: **Numbers → Model**
- Same idea we learned in Week 2.

Tokenizer also handles unequal text input lengths for us

```python
#padding example
texts = [
    "Great campaign!",
    "The campaign uses storytelling to connect with young voters."
]

inputs = tokenizer(
    texts,
    padding=True,
    truncation=True,
    return_tensors="pt"
)

print(inputs)
```

From Text To Model Input (Numbers that Model Can Process)

```
#From text to model input

text = "Journalists are experimenting with generative AI."

inputs = tokenizer(
    text,
    return_tensors="pt"
)

print(inputs)
```

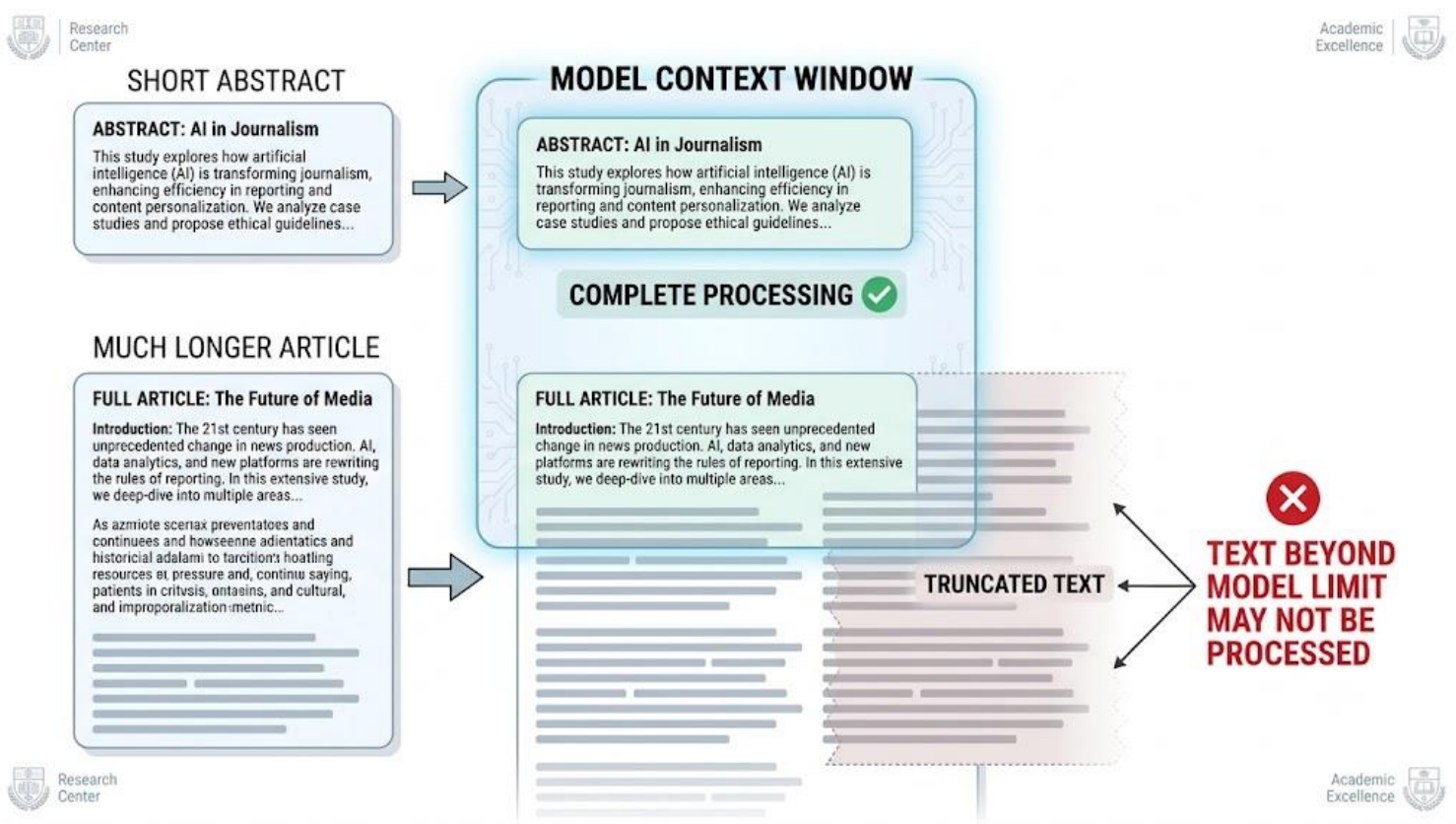
Can We Turn the IDs Back Into Text?

```
#Turn ID back to text
ids = inputs["input_ids"][0]

tokenizer.decode(ids)
```

```
'[CLS] journalists are experimenting with generative ai. [SEP]'
```

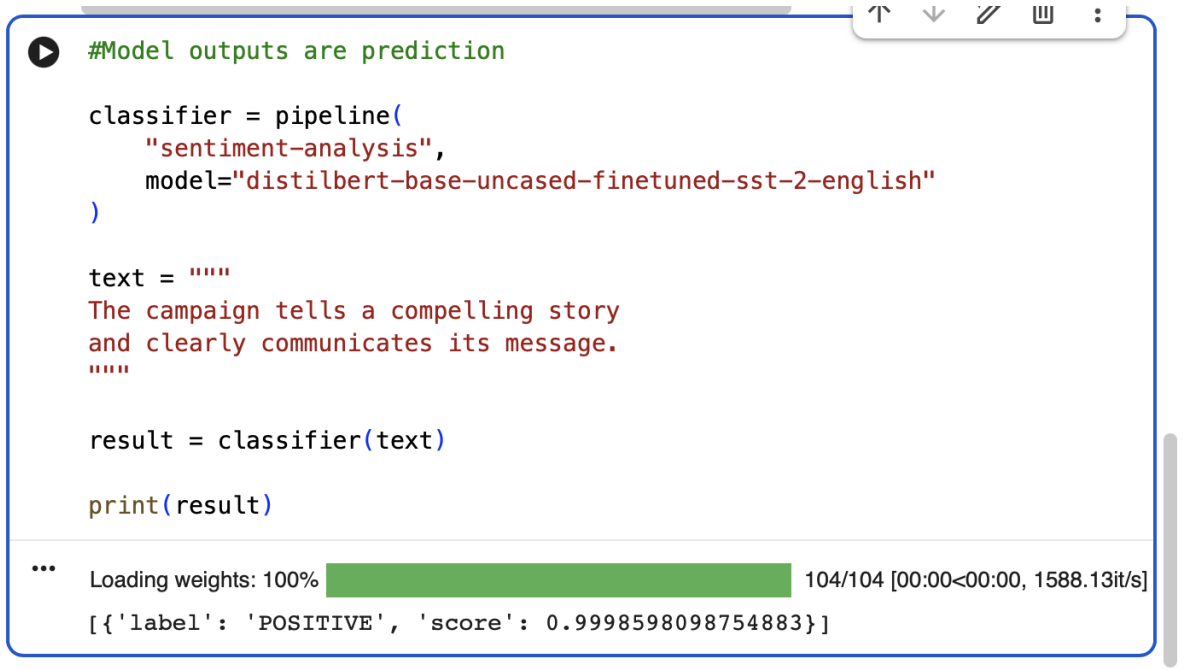
A Model May Not Read the Entire Document

- Models have input limits
- Long documents may exceed them
- Truncation removes text
- Model choice matters
- Always check documentation



Outputs Are Model Predictions

- A Score Is Not “Truth”
- Model says: “**POSITIVE — 99%**”
- This means: **The model is confident in its prediction.**
- It does NOT mean: **The answer is definitely correct.**
- **Researchers still need to check.**



Model Output Depends on the Model's Task

```
#Model outputs depend on model tasks
texts = [
    "The campaign tells a powerful story.",
    "The advertisement is confusing.",
    "The mayor announced the campaign Tuesday."
]

classifier(texts)
```

```
[{'label': 'POSITIVE', 'score': 0.9998776912689209},
 {'label': 'NEGATIVE', 'score': 0.9995278120040894},
 {'label': 'POSITIVE', 'score': 0.9836065769195557}]
```

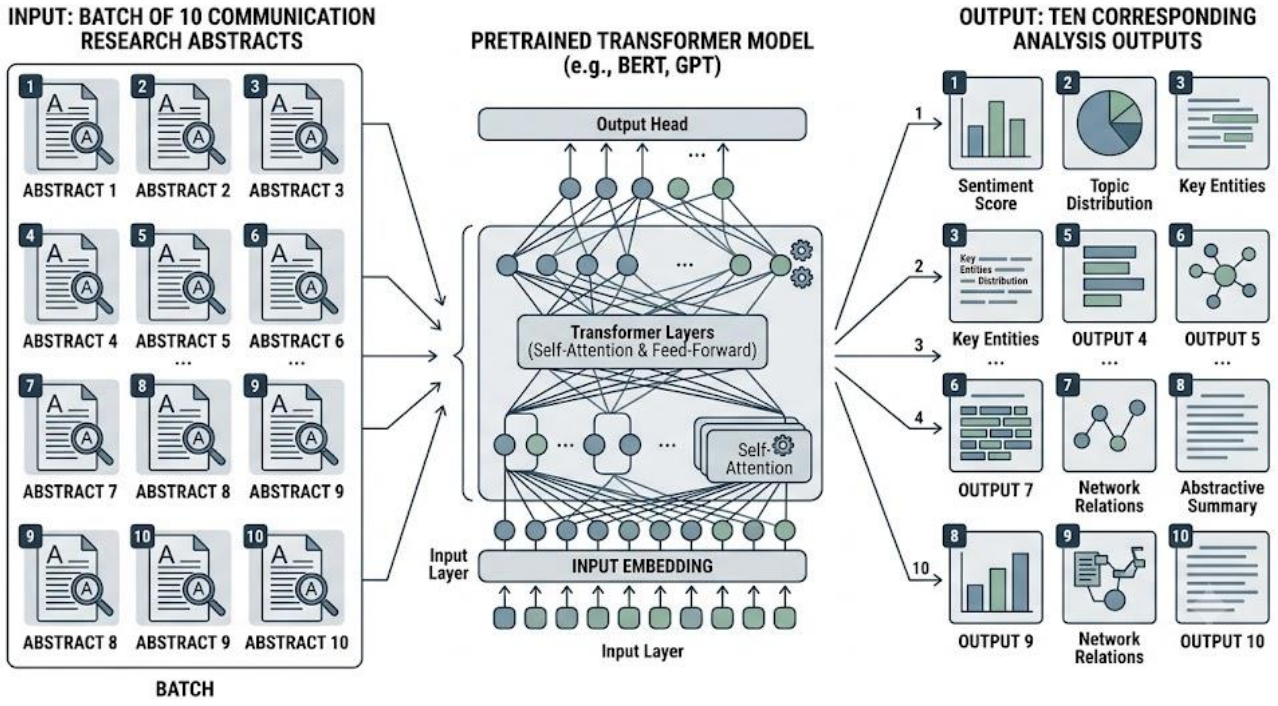
Is neutral even available?

One Document vs. Many Documents

- The idea of processing multiple sequences is called **batching**.
- Instead of manually doing the same action 100 times, we can give the computer multiple texts and systematically apply the model.

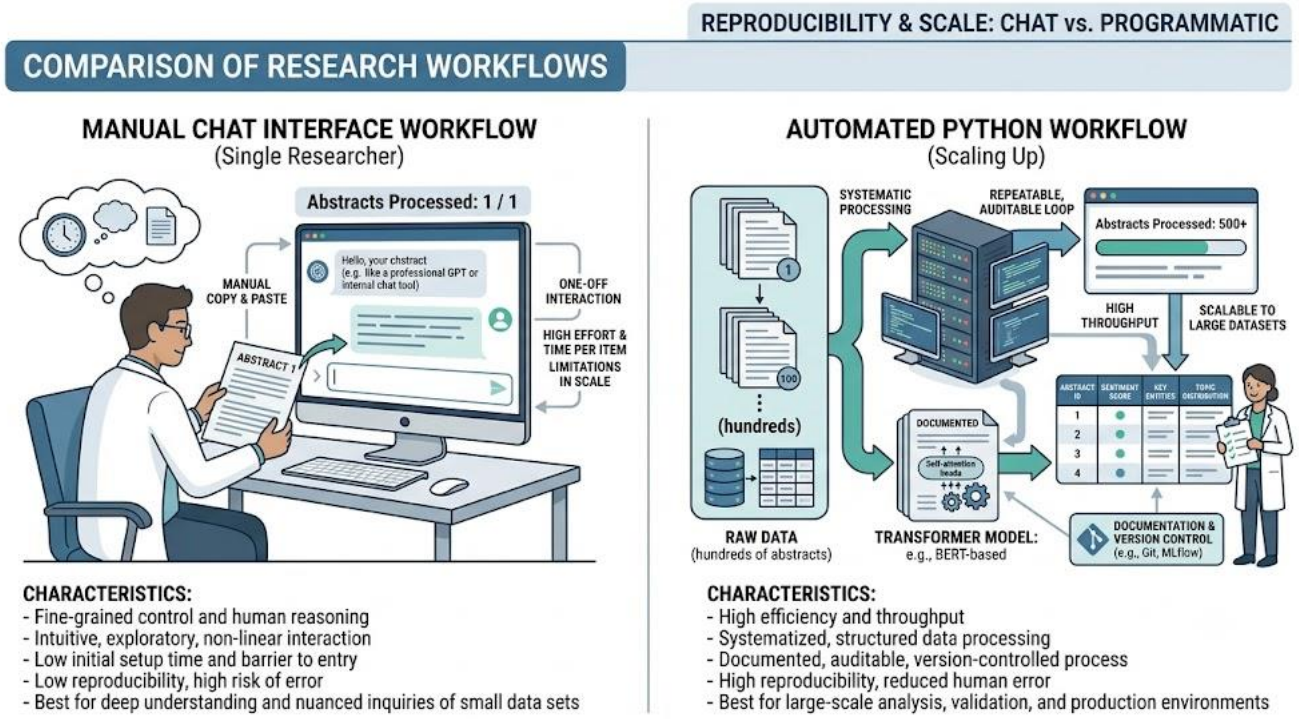
Batching

- **Batch = multiple inputs processed together**
- More efficient
- Same model
- Same procedure
- Multiple outputs



Why LLM-Assisted Literature Review Matters for Research

- Imagine **500 abstracts you need to read for a literature review process**
- You *could* paste 500 abstracts manually into a chatbot. But doing so one at a time would be inefficient, difficult to reproduce, and difficult to store systematically



Next: AI-Assisted Literature Prioritization

- Research question
- Candidate abstracts
- Model-assisted triage
- Human review
- Compare judgments